Universidad Distrital Francisco José de Caldas

School of Engineering

Computer Engineering

MDP Formulation for Reinforcement Learning

Real Estate Investment Decision-Making

Course: Machine Learning — Semester 2026-III
Workshop: No. 3 — Reinforcement Learning (Part B)
Team: Group 1

Authors

Chavez Porras, Miguel Alejandro
Guerrero Ordeñez, Luisa Fernanda
Arboleda Zambrano, Dilan Stive

2026

Contents

1 Problem Statement **3**

1.1 Domain Context . . . 3

1.2 Reformulation as a Sequential Decision Problem . . . 3

1.3 Justification for Reinforcement Learning . . . 4

2 Markov Decision Process Formulation **4**

3 State Space (S) **5**

3.1 State Features . . . 5

3.2 State Dimension . . . 5

3.3 Markov Property . . . 5

4 Action Set (A) **6**

4.1 Handling Invalid Actions . . . 6

4.2 Action Space Type . . . 6

5 Transition Dynamics (P) **6**

5.1 Deterministic Components . . . 7

5.2 Stochastic Components . . . 7

5.3 Cash-Flow Table . . . 7

6 Reward Function (R) **8**

6.1 Per-Step Reward . . . 8

6.2 Terminal Reward . . . 8

6.3 Reward Shaping Rationale . . . 9

6.4 Discount Factor . . . 9

7 Episode Termination **9**

8 MDP Summary **10**

9 Connection with Workshops 1 and 2 **10**

10 References **11**

Abstract

This document specifies the formulation of a Markov Decision Process (MDP) for a sequential real estate investment task. Building on the supervised models developed in Workshops 1 and 2 for housing price prediction, we reformulate the problem as a decision-making process in which an agent learns to buy, hold, renovate, and sell a property over a fixed horizon to maximize cumulative net profit. We define all five MDP components—state space, action set, transition dynamics, reward function, and episode termination—grounded in the real estate domain and calibrated with statistics from the Ames Housing dataset.

1. Problem Statement

1.1 Domain Context

Our assigned problem domain is **real estate valuation**, specifically housing price prediction using the Ames Housing dataset. In Workshop 1, we implemented supervised regression models (linear, tree-based, and ensemble methods) to estimate property prices from structural and locational features. In Workshop 2, we extended this work with deep neural networks, improving predictive performance and capturing non-linear relationships between features.

While these models answer the question *"What is the price of this property?"*, they do not address a fundamentally different and more valuable question for real-world applications: *"What should an investor do with this property over time?"*

1.2 Reformulation as a Sequential Decision Problem

We reformulate the domain as a **reinforcement learning task** where an agent acts as a real estate investor. At each time step (one month), the agent observes the current state of the market and of the property, chooses an action among a discrete set of investment decisions, and receives a reward that reflects the financial consequence of that action.

The agent's objective is to learn a **policy** $\pi(a \mid s)$ that maximizes the expected cumulative discounted reward over a fixed investment horizon. This captures the essential trade-offs of real estate investment:

- **Timing:** buy now or wait for a better price?
- **Value creation:** renovate to increase resale value, or sell as-is?
- **Risk:** hold through market downturns, or liquidate to preserve capital?

1.3 Justification for Reinforcement Learning

Reinforcement learning is the appropriate paradigm for this task for four reasons, following the criteria established in the workshop guidelines:

1. **Sequential decisions:** Every action affects future possibilities. Buying today consumes capital that cannot be deployed tomorrow; holding incurs maintenance costs that reduce cumulative profit.
2. **Reward-based evaluation:** The quality of a decision cannot be assessed in isolation. Selling at a profit depends on the purchase price paid months or years earlier and on market conditions at the time of sale.
3. **Simulable environment:** We can construct a simulator of real estate price dynamics calibrated from historical distributions in the Ames Housing dataset, enabling unlimited agent-environment interaction without ethical or financial risk.
4. **Exploration/exploitation trade-off:** The agent must balance exploiting known profitable strategies (e.g., buy low, sell high) with exploring new ones (e.g., renovation, holding through downturns)—a trade-off that supervised learning cannot represent.

By contrast, supervised learning as used in Workshops 1 and 2 is **sample-efficient** but **static**: it cannot model the delayed consequences of actions, the trade-off between acting and waiting, or policies that depend on the history of states.

2. Markov Decision Process Formulation

We model the investment task as a finite-horizon Markov Decision Process defined by the tuple

$$(S, A, P, R, \gamma, T),$$

where:

- S is the state space;
- A is the action set;
- $P(s' \mid s, a)$ is the transition probability function;
- $R(s, a, s')$ is the reward function;
- $\gamma \in [0, 1]$ is the discount factor;
- T is the episode horizon.

Each component is specified in the following subsections.

3. State Space (S)

The state at time step t is a vector $s_t \in \mathbb{R}^d$ containing all information the agent needs to make an informed decision. It combines market signals, property attributes, and the agent's financial position.

3.1 State Features

Table 1: State-space features

#	Feature	Symbol	Description	Normalized range
1	Current property price	p_t	Estimated market value of the target property	$[0, 1]$
2	Market average price	m_t	Average price of comparable properties in the zone	$[0, 1]$
3	Price trend (3 months)	τ_t	Relative price change over the last three months	$[-1, 1]$
4	Ownership flag	o_t	1 if the agent owns the property, 0 otherwise	$\{0, 1\}$
5	Property condition	c_t	Physical condition index (new = 1, worn = 0)	$[0, 1]$
6	Available capital	k_t	Liquid capital available to the agent	$[0, 1]$
7	Months held	h_t	Number of consecutive months the property has been held	$[0, 1]$
8	Time step	t/T	Normalized position within the episode horizon	$[0, 1]$

3.2 State Dimension

The state dimension is $d = 8$. All features are normalized to a bounded range to stabilize neural network training and ensure numerical consistency across episodes.

3.3 Markov Property

The state is designed to satisfy the Markov property: it contains all information relevant to predicting the next state and expected reward, including market trends, ownership status, and financial position. No history buffer is required for the initial formulation; if performance proves insufficient, we may extend the state with the last k price observations in a refined version.

4. Action Set (A)

The action space is **discrete** with four actions. Discreteness reflects the real-world nature of investment decisions, which are categorical rather than continuous in this simplified setting.

Table 2: Available actions

Index	Action	Interpretation	Valid when
0	`WAIT`	Do nothing this month	Always
1	`BUY`	Purchase the property at current price	$o_t = 0$
2	`RENOVATE`	Invest in renovations to increase property value	$o_t = 1$
3	`SELL`	Sell the property at current market price	$o_t = 1$

4.1 Handling Invalid Actions

Two of the four actions are only valid under specific ownership states. We handle invalid actions in two complementary ways:

1. **Penalty-based (primary):** Invalid actions are allowed but incur a fixed negative penalty in the reward, discouraging the agent from repeating them.
2. **Masking (optional refinement):** The action probability of invalid actions is set to zero during action selection, a standard technique in deep RL for discrete control.

We adopt the penalty-based approach in the baseline implementation for simplicity, and evaluate masking as a refinement.

4.2 Action Space Type

`Discrete(4)` — compatible with Deep Q-Networks (DQN), our chosen agent algorithm.

5. Transition Dynamics (P)

The transition function defines how the environment evolves:

$$s_{t+1} = f(s_t, a_t, \varepsilon_t),$$

where ε_t represents exogenous stochasticity from the market.

5.1 Deterministic Components

The following state variables evolve deterministically given the action:

- **Ownership flag (o_t):** set to 1 after `BUY`, set to 0 after `SELL`, unchanged otherwise.
- **Months held (h_t):** increments by one at each step while the property is owned; resets to 0 upon `SELL`.
- **Available capital (k_t):** updated according to the cash-flow table in Section 5.3.
- **Time step (t/T):** increments by $1/T$ at each step.
- **Property condition (c_t):** decays slowly over time at a fixed rate while owned; increases by a fixed increment upon `RENOVATE`.

5.2 Stochastic Components

Market prices evolve stochastically, capturing the uncertainty inherent to real estate markets:

- **Current property price (p_t):** follows a geometric random walk with drift:

$$p_{t+1} = p_t \exp(\mu + \sigma z_t), \qquad z_t \sim \mathcal{N}(0, 1).$$

  The drift μ and volatility σ are calibrated from the Ames Housing price distribution. We use monthly equivalents of realistic annual parameters ($\mu_{\text{annual}} \approx 3\%$, $\sigma_{\text{annual}} \approx 5\%$).

- **Market average price (m_t):** follows the same stochastic process with lower volatility ($\sigma_{\text{market}} = 0.5\,\sigma_{\text{property}}$), reflecting the fact that market aggregates are smoother than individual properties.
- **Price trend (τ_t):** recomputed deterministically from the last three observed prices.

5.3 Cash-Flow Table

Default parameter values (tunable hyperparameters):

- `buying_cost_rate` $= 0.03$ (3% closing costs)
- `selling_cost_rate` $= 0.05$ (5% agent commission and taxes)
- `renovation_cost` $= 0.08$ (8% of initial price)
- `maintenance_cost` $= 0.002$ (0.2% of initial price per month)

Table 3: Cash-flow effects

Event	**Effect on capital (k_t)**
`BUY`	$-p_t(1 + \text{buying_cost_rate})$
`RENOVATE`	$-\text{renovation_cost}$
`SELL`	$+p_t(1 + \text{condition_bonus}(c_t))(1 - \text{selling_cost_rate})$
`HOLD` (per month)	$-\text{maintenance_cost}$
`WAIT` (per month)	$-\text{idle_capital_penalty}$ (if capital > threshold)

- `idle_capital_penalty` $= 0.001$ (small penalty for inaction with idle capital)
- $\text{condition_bonus}(c) = 0.10c$ (up to 10% price bonus at full condition)

All monetary quantities are expressed as fractions of the initial property price to keep the state space normalized.

6. Reward Function (R)

The reward function communicates to the agent which behaviors lead to the desired outcome. We adopt a **dense + terminal** reward scheme to avoid the slow learning associated with purely sparse (terminal-only) rewards, while keeping the terminal signal dominant.

6.1 Per-Step Reward

At each step, the reward is defined as

$$r_t = \begin{cases} p_t\,\text{condition_bonus}(c_t) - \text{purchase_price} - \text{buying_costs}, & \text{if } a_t = \texttt{SELL}, \\ -\text{maintenance_cost}, & \text{if } a_t = \texttt{HOLD} \text{ and the agent owns the} \\ -\text{renovation_cost} + \text{renovation_bonus}, & \text{if } a_t = \texttt{RENOVATE}, \\ -\text{invalid_action_penalty}, & \text{if } a_t \text{ is invalid}, \\ -\text{idle_capital_penalty}, & \text{if } a_t = \texttt{WAIT} \text{ and } k_t > \text{threshold}, \\ 0, & \text{otherwise.} \end{cases}$$

6.2 Terminal Reward

At the end of an episode ($t = T$), if the agent still owns the property, a **forced-sale penalty** is applied:

$$r_{\text{terminal}} = p_T(1 - \text{forced_sale_cost}) - \text{purchase_price}.$$

This ensures the agent does not hoard the property indefinitely to avoid the cost of selling, and forces it to internalize the full investment cycle.

6.3 Reward Shaping Rationale

Reward shaping is one of the most critical design decisions in this MDP. Our choices are grounded in the following principles:

1. **Dense intermediate signals:** Maintenance costs and renovation bonuses provide frequent feedback, helping the agent learn the value of holding and improving a property.
2. **Idle-capital penalty:** Without this term, the trivial policy "never invest" is a local optimum: the agent preserves capital but never earns profit. A small penalty for inaction breaks this symmetry.
3. **Invalid-action penalty:** Ensures that the agent learns the ownership constraints implicitly, even without action masking.
4. **Dominant terminal signal:** The sale reward and forced-sale penalty dominate cumulative profit, ensuring the agent optimizes the full investment cycle rather than gaming intermediate rewards.
5. **Bounded and normalized:** All reward terms are expressed as fractions of the initial property price, keeping magnitudes in a stable range for neural network training.

6.4 Discount Factor

We use $\gamma = 0.99$, close to 1, reflecting the long-horizon nature of real estate investment. A lower γ (e.g., 0.90) would make the agent myopic and reduce its willingness to hold through market downturns.

7. Episode Termination

An episode terminates when **any** of the following conditions occurs:

1. **Maximum horizon reached:** $t = T$, where $T = 120$ months (10 years). This is the standard termination condition and matches realistic long-term investment horizons.
2. **Bankruptcy:** $k_t < 0$, meaning the agent cannot cover its costs. This is a failure state that the agent should learn to avoid.

3. **Optional success threshold:** $k_t \geq$ target_return (e.g., 2.0, doubling the initial capital). This is used only during evaluation as an early-stop criterion; during training we always run the full horizon to ensure consistent episode lengths.

After termination, the environment resets to a randomized initial state:

- Initial property price: sampled from the Ames Housing price distribution.
- Initial capital: $k_0 = 1.0$ (normalized to initial price).
- Initial ownership: $o_0 = 0$.
- Initial condition: $c_0 \sim \text{Uniform}(0.4, 0.8)$, reflecting typical used-property condition.

8. MDP Summary

Table 4: Summary of the MDP formulation

Component	**Specification**
State space	$\mathbb{R}^8$ — normalized continuous features
Action space	`Discrete(4)` — `WAIT`, `BUY`, `RENOVATE`, `SELL`
Transition	$s_{t+1} = f(s_t, a_t, \varepsilon_t)$; price as geometric random walk
Reward	Dense per-step + terminal; shaped with penalties and bonuses
Discount factor	$\gamma = 0.99$
Horizon	$T = 120$ months
Termination	Horizon reached, bankruptcy, or success threshold
Algorithm	Deep Q-Network (DQN) — discrete actions, Markovian state
Baselines	Random policy, buy-and-hold heuristic, greedy buy-low/sell-high

9. Connection with Workshops 1 and 2

The MDP formulation is not constructed in isolation; it leverages and extends the work of the previous workshops.

From Workshop 1 (Supervised Learning): The price distribution of the Ames Housing dataset is used to calibrate the initial property price and the drift/volatility parameters of the stochastic price process. Feature importance rankings from supervised models inform which property attributes (e.g., overall quality, living area) most influence value, guiding the design of the renovation bonus function.

From Workshop 2 (Deep Learning): The trained neural network from Workshop 2 can be reused as a **price estimator** within the environment simulator, providing more

realistic price evolution than a synthetic random walk. This enables a natural transfer of representation learning from supervised to reinforcement learning.

Comparison framework: Reinforcement learning will be compared against supervised approaches not on predictive error (RMSE) but on **sample efficiency**: how many environment interactions are required for the RL agent to reach a given cumulative profit, compared to how many labeled examples were needed by supervised models in W1/W2 to reach a given RMSE. This comparison is methodologically fair, as recommended in the workshop guidelines.

10. References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

2. Watkins, C. J. C. H., & Dayan, P. (1992). Q-Learning. *Machine Learning*, 8(3–4), 279–292.

3. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

4. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

5. Towers, M., et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv preprint*.

6. Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8.